CineVerse

Cinema Booking System

DBMS Mini Project — 3rd Semester

'& '& '&

Database	Backend	Frontend
MongoDB	Node.js + Express	React + Vite

Date: 28 October 2026

Submitted for: Database Management Systems (DBMS)

Table of Contents

1.	What is this Project?	3
2.	What Problem does it Solve?	4
3.	Tools & Technologies Used	5
4.	How the System Works (Architecture)	6
5.	Database — What Data We Store	7
6.	How Tables are Related	8
7.	Features of CineVerse	9
8.	How Reservations and Tickets Stay in Sync	10
9.	All API Endpoints (Backend Routes)	11
10.	All Pages in the App	12
11.	How We Keep User Data Private	13
12.	How to Run this Project	14
13.	Sample Data in the Database	15
14.	What We Learned / Conclusion	16

1. What is this Project?

CineVerse is an online cinema booking web app — similar to BookMyShow — that we built as a DBMS mini project. It lets a user browse movie shows, reserve a seat, buy a ticket, order snacks, and use discount coupons, all from a single website.

The project has two main parts:

- **Backend** — the server that stores and manages all data in MongoDB
- **Frontend** — the website that the user sees and interacts with in a browser

er

Why did we build this?

As a DBMS project, our goal was to design a real database, connect it to a backend server, and show how data is stored, retrieved, updated, and deleted. A cinema booking system is a great example because it has multiple types of data (movies, seats, tickets, users) that are all connected to each other — just like in a real application.

7

DB Collections

14

Web Pages

40+

API Routes

1

Locked User

2. What Problem does it Solve?

Booking a cinema ticket manually (calling the theater, going in person) is slow and error-prone. People can accidentally book the same seat twice, or lose their ticket info. CineVerse solves this by storing everything in a database and making sure the data is always correct and up to date.

Problems CineVerse Fixes

Problem: Double Booking
- **Solution:** Two people cannot book the same seat — the system checks before confirming

Problem: Lost Tickets
- **Solution:** All tickets are saved in the database and can be viewed anytime

Problem: No Sync Between Records
- **Solution:** When you reserve a seat, a ticket is also created automatically — they always match

Problem: No Privacy
- **Solution:** Each user only sees their own bookings and tickets, not anyone else's

Problem: Manual Discount Calculation
- **Solution:** Coupons are applied automatically and the final price is calculated by the backend

Who uses this App?

For this project, there is one fixed user — Arjun Sharma (CUST-001). He is automatically loaded when you open the app. There is no login page because this is a demo project focused on the database part, not authentication.

3. Tools & Technologies Used

Here are all the technologies we used and a simple explanation of what each one does:

Technology	What it does in simple words
MongoDB	Database — stores all our data (users, tickets, shows, etc.) as documents (like JSON files)
Mongoose	Helps Node.js talk to MongoDB — also lets us define the shape/structure of our data
Node.js	Runs JavaScript on the server (backend) instead of in the browser
Express.js	A simple tool that helps us create API routes like /api/tickets, /api/bookings, etc.
React 18	Used to build the website UI — everything you see on screen is made with React
Vite	Starts the React app quickly during development (like a fast local web server)
React Router	Handles navigation between pages (e.g., clicking "My Tickets" goes to /tickets)
Axios	Sends HTTP requests from the React app to the backend (like fetch but simpler)
Tailwind CSS	Styles the website — gives us the dark theme, gold colors, and card layouts
PDFKit	Used to generate this PDF report from a Node.js script

Why MongoDB instead of SQL (MySQL)?

In SQL databases, data is stored in fixed tables with strict rules. MongoDB stores data as flexible documents (similar to JSON), which is better for a project like this where different shows can have different details. Also, MongoDB is very popular in real-world industry projects, so learning it is useful.

4. How the System Works (Architecture)

CineVerse has 3 layers. Think of it like a restaurant: the customer (frontend), the waiter (backend API), and the kitchen (database).

Layer 1 — Frontend (React)

http://localhost:5173

The website the user sees. Built with React + Vite. When you click buttons, it sends requests to the backend.

%¼

Layer 2 — Backend API (Express)

http://localhost:5001

The server. It receives requests, runs the business logic (e.g., sync bookings with tickets), and talks to MongoDB.

%¼

Layer 3 — Database (MongoDB)

mongodb://localhost:27017/cinemadb

Stores all the actual data — customers, shows, bookings, tickets, snacks, coupons, theaters.

How a Request Flows (Example: Reserve a Seat)

Step 1:
User fills the "Reserve Seat" form and clicks

Submit on the website

Step 2: React (frontend) sends a POST request to http://localhost:5001/api/bookings

Step 3: Express (backend) receives the request and validates the data

Step 4: Backend creates a Booking record in MongoDB

Step 5: Backend also creates a matching Ticket record (automatically)

Step 6: Backend sends back a success response with the new booking data

Step 7: React shows a success message and updates the page

5. Database — What Data We Store

Our MongoDB database (called "cinemadb") has 7 collections. A "collection" in MongoDB is like a table in SQL. Each item in a collection is called a "document" (like a row in SQL). Every collection has a custom ID like CUST-001, TKT-001, etc. to make it easier to read.

customers — Who is using the app

Field Name	Data Type / Example	What it means
customerId	CUST-001, CUST-002…	Our own readable ID
name	Text	Full name of the user
email	Text (unique)	Email address
phone	Text	Phone number
address	Text	Home address
memberSince	Date	When they joined

theaters — Where movies are shown

Field Name	Data Type / Example	What it means
theaterId	THR-001…	Readable ID
name	Text	Theater name (e.g., PVR IMAX)
location	Text	City and area
totalSeats	Number	Total seats in the hall
facilities	List of Text	E.g., Dolby, 4K Screen, Recliner

shows — The movie screenings

Field Name	Data Type / Example	What it means
showId	SHW-001…	Readable ID
movieTitle	Text	Name of the movie
theater	Reference !' Theater	Which theater is showing it
showTime	Date & Time	When the show starts
ticketPrice	Number ([1])	Base ticket price
genre	Text	Action, Drama, Horror, etc.
language	Text	Hindi, Tamil, English, etc.
availableSeats	Number	How many seats are left
rating	Text	U / UA / A certification

bookings — Seat reservations

Field Name	Data Type / Example	What it means
bookingId	BKG-001…	Readable ID
customer	Reference !’ Customer	Who made this booking
show	Reference !’ Show	Which show was booked
seatNumber	Text	Specific seat (e.g., A-12)
bookingDate	Date	When they booked it
status	Text	confirmed / cancelled / pending
totalAmount	Number (¹)	How much they paid

tickets — Actual movie tickets

Field Name	Data Type / Example	What it means
ticketId	TKT-001…	Readable ID
customer	Reference !’ Customer	Ticket owner
show	Reference !’ Show	Show for this ticket
booking	Reference !’ Booking	Linked reservation (auto-set)
seatNumber	Text	Seat assigned
purchaseDate	Date	When the ticket was bought
price	Number (¹)	Final price after coupon discount
couponApplied	Text	Coupon code used (if any)
discountAmount	Number (¹)	How much discount was applied
snacks	Reference !’ Snack	Linked snack order (if any)

snacks — Food and drink orders

Field Name	Data Type / Example	What it means
snackId	SNK-001…	Readable ID
customer	Reference !’ Customer	Who ordered
items	List of {name, qty, price}	Items in the order
totalPrice	Number (¹)	Total food bill
orderDate	Date	When the order was placed

coupons — Discount codes

Field Name	Data Type / Example	What it means
couponId	CPN-001…	Readable ID
code	Text	The coupon code (e.g., FIRST50)
discountType	Text	"percentage" or "flat"
discountValue	Number	How much discount (% or ¹)
minPurchase	Number (¹)	Minimum order needed to use it
maxDiscount	Number (¹)	Maximum discount allowed
validUntil	Date	Expiry date
isActive	True/False	Is it still usable?

6. How Tables are Related (Relationships)

In our database, the 7 collections are connected to each other. Think of it like a web — if you pull one thread, it connects to others. In MongoDB, we use "references" (storing the ID of another document) to create these connections. The table below shows all the relationships:

Collection A	Collection B	Relationship	Meaning in simple words
Customer	Booking	One to Many	One person can make many reservations
Customer	Ticket	One to Many	One person can have many tickets
Booking	Ticket	One to One	Every reservation has exactly one ticket (they are always in sync)
Show	Booking	One to Many	One movie show can have many reservations (up to seat limit)
Show	Ticket	One to Many	One show can have many tickets bought for it
Theater	Show	One to Many	One theater can screen many different movies
Ticket	Snack	One to One	A ticket can optionally have one snack order linked to it
Ticket	Coupon	Many to One	Many tickets can use the same coupon code

Important: How we handle this in MongoDB

SQL databases have built-in "Foreign Key" rules that automatically reject invalid references. MongoDB does NOT have this by default. So we handle it ourselves in the backend code:

- When a booking is deleted, our code also deletes the matching ticket
- W

he
n
a ti
ck
et
is
del
ete
d,
ou
r c
od
e a
lso
de
let
es
the
m
atc
hin
g b
oo
kin
g

- When data is fetched, our code checks for "orphan" records and fixes them automatically
- Before

creating a new ticket, we check if one already exists (to avoid duplicates)

7. Features of CineVerse

01. Browse Shows Ø<ß- /api/shows

What it does:
Shows all available movies with their time, price, theater, language and genre. You can filter by genre or language.

How it works:
Frontend calls GET /api/shows. The backend fetches all shows from MongoDB and also fills in the theater name (using .populate()).

02. Reserve a Seat Ø=Ü¦ /api/bookings/new

What it does:
User picks a show and enters a seat number to reserve it. A reservation (Booking) is saved in the database.

How it works:
Frontend sends POST /api/bookings. Backend saves the booking AND automatically creates a matching ticket. Available seat count on the show is reduced by 1.

03. My Reservations Ø=ÜÈ /api/bookings

What it does:
Shows all the reservations made by the current user. User can click to see details or delete a reservation.

How it works:
GET /api/bookings?customerId=... fetches only this user's bookings. Deleting also removes the linked ticket automatically.

04. My Tickets Ø<ßa /tickets

What it does:
Shows all tickets the user has. Tickets are always in sync with reservations — they always show the same data.

How it works:
GET /api/tickets?customerId=... fetches only this user's tickets. Deleting also removes the linked booking.

05. Buy a Ticket Directly Ø<ßÝp /tickets/new

What it does:
User can also buy a ticket directly (without first going to "Reserve Seat"). Optional snack selection is shown too.

How it works:
POST /api/tickets. Backend creates the ticket AND a matching booking automatically.

06. Snack Bar Ø<ß» /snacks

What it does:
User can browse cinema snack items (popcorn, drinks, nachos, etc.) and place an order. Orders are linked to tickets.

How it works:
GET /api/snacks shows the menu. POST /api/snacks saves the order to MongoDB under the user's account.

07. Deals & Offers (Coupons) Ø<ß÷ /coupons

What it does: All active discount coupons are shown as voucher cards. User can copy a coupon code with one click.

How it works:
GET /api/coupons fetches all active coupons from the DB. The coupon is applied on the ticket detail page.

08. Apply Coupon on Ticket Ø=Ü /tickets/:id

What it does: On the ticket detail page, user can type in a coupon code. The discount is calculated and the final price is updated.

How it works: POST /api/tickets/:id/coupon validates the coupon, calculates discount, and updates the ticket's price in MongoDB.

09. My Profile Ø=Üd /profile

What it does: Shows the user's personal info (name, email, phone, address) and a summary of their activity.

How it works: Loads from UserContext which fetches the first customer from GET /api/customers. No login required.

8. How Reservations and Tickets Stay in Sync

This was the most important and tricky part of the project. Every booking (reservation) should always have exactly one ticket — and every ticket should always have exactly one booking. They must always match. Here is exactly how we did it:

When you CREATE a Reservation:

1. User fills the "Reserve Seat" form and submits it
2. Frontend sends: POST /api/bookings (only one request — no double calls)
3. Backend saves the Booking in MongoDB
4. Backend checks: "Does a ticket already exist for this show + seat + user?"
5. If NO !' Backend creates a new Ticket automatically
6. If YES !' Backend skips (to avoid creating 2 tickets)
7. Available seats on the Show are reduced by 1

When you DELETE a Reservation:

1. User clicks "Delete" on a reservation
2. Frontend sends: DELETE /api/bookings/:id
3. Backend finds the Booking in MongoDB
4. Backend finds ALL Tickets for this show + seat + user and deletes them
5. Backend then deletes the Booking itself
6. Available seats on the Show are increased by 1 (seat is freed)

Same thing works in reverse (when you delete a Ticket):

If the user deletes from "My Tickets" instead, the same logic runs in the opposite direction — the matching Booking is also deleted. Both pages always stay in sync no matter which one you use.

Auto-Fix (Auto-Heal):

Every time you open "My Reservations" or "My Tickets", the backend does a quick check in the background:

%If a ticket exists but its matching booking is missing !' booking

is
cre
ate
d a
uto
ma
tic
all
y
%If
a b
oo
kin
g e
xis
ts
but

its
ma
tch
ing
tic
ket
is
mi
ssi
ng
!’ tic
ket
is
cre
ate
d a
uto
ma
tic
all
y
This makes sure the data is always consistent, even if something went wrong before.

9. All API Endpoints (Backend Routes)

An "API endpoint" is basically a URL on the backend server that does something when you call it. The frontend calls these URLs to get or send data. Here are all the routes we built:

Method	URL	What it does
GET	/api/customers	Get the list of all customers
GET	/api/customers/:id	Get one customer by their ID
GET	/api/shows	Get all movie shows (with theater name filled in)
GET	/api/shows/:id	Get details of one specific show
POST	/api/shows	Add a new show to the database
PUT	/api/shows/:id	Update show info (e.g., change price)
DELETE	/api/shows/:id	Delete a show
GET	/api/theaters	Get all theaters
GET	/api/theaters/:id	Get one theater's details
GET	/api/bookings	Get all bookings for this user (also auto-fixes orphans)
GET	/api/bookings/:id	Get one specific booking
POST	/api/bookings	Create a booking !' also auto-creates a ticket
PUT	/api/bookings/:id	Update booking status
DELETE	/api/bookings/:id	Delete booking !' also deletes the matching ticket
GET	/api/tickets	Get all tickets for this user (also auto-fixes orphans)
GET	/api/tickets/:id	Get one specific ticket
POST	/api/tickets	Create a ticket !' also auto-creates a booking
PUT	/api/tickets/:id	Update ticket info
DELETE	/api/tickets/:id	Delete ticket !' also deletes the matching booking
POST	/api/tickets/:id/coupon	Apply a coupon code to a ticket
DELETE	/api/tickets/:id/coupon	Remove a coupon from a ticket
GET	/api/snacks	Get this user's snack orders
GET	/api/snacks/:id	Get one snack order
POST	/api/snacks	Place a new snack order
DELETE	/api/snacks/:id	Cancel a snack order
GET	/api/coupons	Get all active coupon codes
GET	/api/coupons/:code	Check if a coupon code is valid

10. All Pages in the App

The React frontend has 14 pages. Every page is a separate React component. React Router decides which page to show based on the URL in the browser.

URL Route	Page Name	What the user can do here
/	Home / Landing	Welcomes the user, shows featured movies and highlights
/shows	Browse Shows	See all movies, filter by genre & language, click to view details
/shows/:id	Show Details	See full info about one movie show
/bookings	My Reservations	See all seat reservations, delete them
/bookings/new	Reserve a Seat	Fill a form to reserve a seat for a show
/bookings/:id	Reservation Details	View details of one specific reservation
/tickets	My Tickets	See all tickets, delete them
/tickets/new	Buy a Ticket	Buy a ticket directly (with optional snacks)
/tickets/:id	Ticket Details	View ticket, apply coupon, see snack order
/snacks	Snack Bar	Browse snack menu and see your orders
/snacks/new	Order Snacks	Place a new snack order
/snacks/:id	Snack Order Details	View details of a snack order
/coupons	Deals & Offers	See all coupons, copy codes with one click
/profile	My Profile	See your name, contact info, and activity summary

11. How We Keep User Data Private

Even though this is a simple project with only one user, we still built proper privacy checks into every part of the system. This shows good coding practice and is important if the app ever has multiple users.

Single Locked User

The app always loads the first customer from the database and uses that as the current user. There is no login page or user switching. This keeps things simple for the demo.

Ownership Check on Detail Pages

When you visit a Booking, Ticket, or Snack detail page, the backend checks if the item belongs to the current user. If it does not, you get an "Access Denied" message. This means even if you manually type another booking's URL, you cannot see it.

List Filtering by User

The booking, ticket, and snack lists only show records that belong to the current user. The backend filters using: customer = currentUserId.

No Other Users' Data Shown

The home page does not show total ticket counts, revenue, or any data that could reveal other users' activity.

String Comparison Fix

MongoDB IDs are objects (ObjectId type). When comparing them, you have to convert both sides to strings first. We use String(id) everywhere to avoid bugs where the IDs look equal but fail the check.

Redirect from /customers

If someone navigates to the /customers page (which would show all users), they are automatically redirected to /profile instead.

12. How to Run this Project

Requirements (install these first if not already done)

- Node.js (version 18 or above) — download from nodejs.org
- MongoDB — running locally on your computer (port 27017)
- npm — comes with

No
de.
js
aut
om
ati
cal
ly

Step 1: Seed (fill) the Database with Sample Data

Run this once to add movies, shows, tickets, coupons etc. into MongoDB:

```
cd /path/to/cinema-db/server
node src/seed.js
# This creates all sample data (1 user, 10 shows, 7 tickets, 5 coupons…)
```

Step 2: Start the Backend Server

```
cd /path/to/cinema-db/server
node src/index.js
```

You should see: "Cinema DB Backend Server Running on Port: 5001" and "MongoDB Successfully connected"

Note: We use port 5001 because on Mac, port 5000 is reserved by the operating system.

Step 3: Start the Frontend (React App)

Open a NEW terminal window for this:

```
cd /path/to/cinema-db/client
npm run dev
```

Step 4: Open the App

http://localhost:5173

Open the above URL in any web browser (Chrome, Firefox, Edge). The CineVerse app will load automatically with Arjun Sharma logged in.

13. Sample Data in the Database

The One User (Locked)

Field	Value
Customer ID	CUST-001
Name	Arjun Sharma
Email	arjun.sharma@email.com
Phone	9876543210
Address	Koramangala, Bengaluru
Member Since	January 2023

5 Sample Theaters

Theater ID	Name	City
THR-001	PVR IMAX	Bengaluru
THR-002	INOX Cinemas	Mumbai
THR-003	Cinepolis	Hyderabad
THR-004	PVR Phoenix	Delhi
THR-005	SPI Sathyam	Chennai

Sample Movie Shows

Show ID	Movie	Price	Language	Seats Left
SHW-001	Kalki 2898 AD	₹550	Telugu	180
SHW-002	Stree 2	₹320	Hindi	95
SHW-003	Animal	₹280	Hindi	220
SHW-004	Jawan	₹450	Hindi	50
SHW-005	Leo	₹380	Tamil	130

Sample Coupons

Code	Type	Discount	Min. Purchase	Works until
FIRST50	Percentage	50% off	₹200	31 Dec 2025
WEEKEND20	Percentage	20% off	₹300	31 Dec 2025
FLAT100	Flat	₹100 off	₹400	31 Mar 2026
SNACK30	Percentage	30% off	₹150	28 Feb 2026
IMAX200	Flat	₹200 off	₹500	30 Nov 2025

14. What We Learned / Conclusion

CineVerse is a complete, working full-stack web application that covers all the topics taught in a DBMS course — designing a database, writing queries, connecting tables (or collections), and making sure data stays correct at all times.

What we learned from building this project:

- How to design a database with multiple collections and connect them using references
- How to use MongoDB and Mongoose to store an

d query real data

- How to write backend API routes using Node.js and Express
- How to connect a React frontend to a backend API using Axios
- How to handle data int

egrity without SQL foreign keys — using code-level logic
- How "cascading delete" works — deleting one record automatically deletes related records
- How to build a real-w

orl
d f
eat
ur
e (l
ike
se
at
res
er
vat
ion
) e
nd
-
to-
en
d f
ro
m
dat
ab
as
e
to
UI

%3 H
ow
to
pr
ote
ct
us
er
dat
a u
sin
g o
wn
ers
hip
ch
ec
ks
in
AP
I r
out
es

Challenges we faced:

'L Problem: Duplicate Ticket Bug

' Fix: At first, the frontend was calling "create ticket" AND the backend was also

creating one auto matically — so 2 tickets ap peared per booking. Fixed by removing the frontend call and letting only the backend handle it.

'L Problem: Port 5000 Conflict on Mac

' Fix: macOS uses port 5000 for its own services. We had to change our backend port to 5001.

'L Problem: MongoDB ID Comparison

' Fix: MongoDB

ObjectIds are objects, not strings. C omparing them with === was giving wrong results. Fixed using String() c onversion on both sides.

'L Problem: Keeping Sync on Both Sides

' Fix: Making sure deleting from "My Tickets" also deleted

from "My Reservations" (and the other way around) required careful backend logic on both delete routes.

CineVerse — Your Seat. Your Story.

MongoDB · Express.js · React 18 · Node.js · Vite

DBMS Mini Project | 3rd Semester | 2nd Year Engineering